

Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry

Xueying Du
Fudan University
Shanghai, China
21210240012@m.fudan.edu.cn

Jiayi Feng
Fudan University
Shanghai, China
23210240148@m.fudan.edu.cn

Yi Zou
Fudan University
Shanghai, China
yzou21@m.fudan.edu.cn

Wei Xu
Tencent
Shanghai, China
davidxu@tencent.com

Jie Ma
Tencent
Shanghai, China
jima@tencent.com

Wei Zhang
Tencent
Shanghai, China
marszhang@tencent.com

Sisi Liu
Tencent
Shanghai, China
rainnyliu@tencent.com

Xin Peng
Fudan University
Shanghai, China
pengxin@fudan.edu.cn

Yiling Lou
Fudan University
Shanghai, China
yilinglou@fudan.edu.cn

Abstract

Static analysis tools (SATs) are widely adopted in both academia and industry for improving software quality, yet their practical use is often hindered by high false positive rates, especially in large-scale enterprise systems. These false alarms demand substantial manual inspection, creating severe inefficiencies in industrial code review. While recent work has demonstrated the potential of large language models (LLMs) for false alarm reduction on open-source benchmarks, their effectiveness in real-world enterprise settings remains unclear. To bridge this gap, we conduct the first comprehensive empirical study of diverse LLM-based false alarm reduction techniques in an industrial context at Tencent, one of the largest IT companies in China. Using data from Tencent's enterprise-customized SAT on its large-scale Advertising and Marketing Services software, we construct a dataset of 433 alarms (328 false positives, 105 true positives) covering three common bug types. Through interviewing developers and analyzing the data, our results highlight the prevalence of false positives, which wastes substantial manual effort (e.g., 10 - 20 minutes of manual inspection per alarm). Meanwhile, our results show the huge potential of LLMs for reducing false alarms in industrial settings (e.g., hybrid techniques of LLM and static analysis eliminate 94–98% of false positives with high recall). Furthermore, LLM-based techniques are cost-effective, with per-alarm costs as low as 2.1–109.5 seconds and \$0.0011–\$0.12, representing orders-of-magnitude savings compared to manual review. Finally, our case analysis further identifies key limitations of LLM-based false alarm reduction in industrial settings.

CCS Concepts

• **Software and its engineering** → *Maintaining software*; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Static Bug Detection, Large Language Models, False Positives

ACM Reference Format:

Xueying Du, Jiayi Feng, Yi Zou, Wei Xu, Jie Ma, Wei Zhang, Sisi Liu, Xin Peng, and Yiling Lou. 2026. Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-SEIP '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3786583.3786910>

1 Introduction

Static analysis tools (SATs) play a crucial role in ensuring software security, reliability, and maintainability. They scan codebase without extensive test execution [18–20, 49, 50, 52, 55, 62], facilitating early bug explosion during the software development process. To date, both academia and industry have proposed many static analysis tools (e.g., Github CodeQL [5], CppCheck [7], Facebook Infer [9]), which are able to detect diverse bug types (e.g., Null-Pointer-Dereference and Use-After-Free) and have been widely adopted in practice [23, 28, 59].

However, SATs often suffer from high false positive rates when applied to large-scale software systems in practice. For example, previous studies [23, 59] show that state-of-the-art open-source SATs (e.g., CodeQL and Infer) exhibit over 95% false alarm rates when detecting Null Pointer Dereference (NPD) bugs in large-scale projects such as the Linux Kernel. This limitation arises from the need to over-approximate program behaviors to guarantee soundness, producing spurious warnings along infeasible paths. The underlying models of SATs typically abstract away complex features like pointer aliasing, external libraries, and conditional compilation, leading to mismatches with real-world behavior. Consequently, especially under heightened security requirements, the trade-off



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-SEIP '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2426-8/26/04
<https://doi.org/10.1145/3786583.3786910>

between soundness and precision favors conservative reporting, resulting in warnings that are formally justifiable but practically infeasible, ultimately overwhelming developers with false alarms.

In practice, bug alarms reported by SATs would be manually inspected by developers to determine whether they correspond to true bugs or false positives. Such manual inspection is labor-intensive and time-consuming. Our survey within Tencent (i.e., one of the largest IT companies in China) reveals that developers spend on average more than 10 minutes reviewing one single static bug alarm, with many alarms even requiring multiple rounds of inspection. Moreover, given the large scale of enterprise software codebase, one single scan can produce hundreds of alarms. Therefore, the large volume of false positives leads to significant waste of manual effort, highlighting the necessity of automatic false alarm reduction techniques.

Traditional false alarm reduction techniques rely on symbolic execution [26, 34] or model checking [46, 54], which suffer from scalability and generalization issues when applied to large-scale enterprise software systems. Learning-based false positive reduction techniques train neural networks on existing false and true positives, which require massive labeled training data for each bug type and thus have limited generalizability across different bug types or for an enterprise where training data is unavailable. Recently, given the rapid development of Large Language Models (LLMs), some work [23, 59] proposes to reduce false alarms with LLMs, and their experimental results have revealed the potential of LLMs in reducing false alarms reported by open-source SATs on synthetic and open-source projects (e.g., Juliet [10] and Gawk [8]). Nevertheless, **it still remains unclear how LLM-based false alarm reduction techniques perform in industry, i.e., for real-world proprietary enterprise software**. In particular, (1) *proprietary enterprise software* differs significantly from *open-source software*, especially for the code characteristics, development models, and code review practices. For instance, enterprise codebases typically enforce strict internal standards and follow rapid DevOps driven iterations, whereas open-source projects adopt more flexible, community-defined norms and evolve at a slower pace. Moreover, for code review practices, open-source projects benefit from extensive peer reviews by diverse contributors, while proprietary enterprises tend to have more centralized, less frequent reviews. These differences may impact the types and distributions of bugs in each environment. (2) *Enterprise-customized SATs* also differ a lot from open-source SATs (e.g., CodeQL). In fact, existing open-source SATs often fall short of meeting the comprehensive and customized needs of enterprises, particularly in terms of efficiency, workflow integration, security, and cost. As a result, enterprises often develop customized SATs tailored to their specific needs and software. The performance differences between these enterprise-developed and open-source tools can significantly influence the effectiveness of false positive reduction.

To fill the knowledge gap, in this work, we perform an extensive study to empirically investigate how LLMs perform in reducing static false alarms for large-scale industrial software systems at Tencent [12]. Tencent is one of the largest IT companies in China, which covers diverse businesses such as social media, digital entertainment, and cloud computing with billions of active users [13, 14]. We collect data from the real-world static bug detection pipeline at

Tencent, which includes the bug alarms reported by its enterprise-customized SAT (i.e., BkCheck) on the large-scale software of its Advertising and Marketing Services (AMS) business line (i.e., over hundreds of components). Based on the historical data, we construct a real-world industry false alarm reduction dataset of 433 bug alarms (i.e., 328 false positives and 105 true positives), which covers the three most frequent bug types, including Null Pointer Dereference (NPD), Out-of-Bounds (OOB), and Divide-by-Zero (DBZ) reported by BkCheck in Tencent. Based on our dataset, we empirically study the performance of LLMs in reducing false alarms for industrial software by answering the following research questions.

RQ1 (Prevalence of false positives in industry). We first perform interviews with Tencent developers to investigate the application practice of SATs. SATs serve as a critical gated check-in during the code review process at Tencent, and all the reported bug alarms would be manually reviewed by developers, with false alarms proceeding to a second-round manual validation to avoid missing any potential bugs. False alarms are not only prevalent (i.e., over 76%), but also require massive manual inspection effort (i.e., on average 10 - 20 minutes per alarm).

RQ2 (Effectiveness of LLMs in reducing false alarms). We further evaluate a diverse range of different LLM-based false alarm reduction techniques on Tencent datasets, including (1) basic LLMs, (2) different advanced prompting strategies, (3) hybrid techniques of static analysis and LLMs. Moreover, we also compare LLM-based techniques with traditional learning-based techniques on Tencent datasets. *To the best of our knowledge, this is the most comprehensive evaluation on learning-based false alarm reduction of industrial software*. Our experimental results show the strong potential of LLM-based techniques in reducing false positives for industrial software, e.g., LLMPFA, the hybrid techniques of static analysis and LLMs, eliminates 94%–98% of false positives across different backbone LLMs while maintaining high recall.

RQ3 (Cost analysis). We measure the time and money costs of LLM-based false alarm reduction techniques when applied on Tencent datasets. In particular, the average time cost per alarm ranges from 2.1 to 109.5 seconds, and the average expense per alarm is 0.0011\$–0.12\$. Such acceptable time and money costs make LLM-based techniques especially valuable for supporting static false positive reduction in industrial practice, substantially reducing manual inspection effort (i.e., 10 - 20 minutes).

RQ4 (Breakdown and case analysis). We further perform breakdown analysis and find the effectiveness variety of LLM-based false alarm reduction techniques across different bug categories. In particular, we find that most LLM-based techniques perform best on Divide-by-Zero (DBZ) bugs, while their performance is lowest on Null Pointer Dereference (NPD) bugs. Moreover, we perform bad case analysis to identify the limitations of LLMs when applied to enterprise-scale bug detection, such as handling bugs of long code contexts, reasoning complex cascaded constraints, and insufficient semantic understanding. These challenges remain future directions for advancing the effectiveness of LLMs-based false alarm reduction in industry.

To summarize, this paper makes the following contributions:

- **Industrial Investigation.** We interview developers and analyze real-world data within Tencent (one of the largest IT companies in China). Our analysis highlights the prevalence

and serious impact from false positives reported by SATs in industrial settings.

- **Extensive Comparison.** We conduct the first empirical study to extensively investigate LLM-based false alarm techniques in *industrial settings*. Our experiments cover a wide range of LLM-based techniques, including traditional learning-based ones, basic LLMs, advanced prompting strategies, and hybrid techniques of LLMs and static analysis. We systematically evaluate both the effectiveness and costs of LLM-based techniques in 433 real-world bug alarms collected from Tencent. While the dataset cannot be released due to confidentiality, all methods and detailed results are available in our replication package [11].
- **Insights and Implications.** We perform both qualitative and quantitative analysis to show both the promise and limitations of LLM-based false alarm techniques in industry. Moreover, we further discuss practical implications for both practitioners and researchers.

The remainder of this paper is organized as follows. Section 2 presents related work. Section 3 introduces the dataset and the detailed research settings. Section 4 presents the results and analysis. Section 5 discusses our findings and presents implications for both practitioners and researchers. Section 6 highlights the threats to the validity of the study. Finally, Section 7 concludes the paper.

2 Related Work

2.1 Enhancing LLMs in Vulnerability Detection

2.1.1 LLM-based Vulnerability Detection. The majority of existing work focuses on prompt engineering [61, 66], such as chain-of-thought [58, 64] and few-shot learning [17], to facilitate more powerful LLM-based vulnerability detection. Additionally, recent work explores fine-tuning approaches [43, 48, 63] to enhance LLMs in vulnerability detection. Furthermore, recent effort [24] explores using historical vulnerability knowledge for retrieval-augmented generation (RAG) to enhance LLMs' vulnerability detection capabilities. Beyond these, further studies investigated different integration strategies to combine LLMs and static analysis to build more powerful static bug detection techniques [32, 33, 39, 53, 60]. Li et al. [32] propose a framework that integrates UBITect with LLMs to detect Use-Before-Initialization (UBI) bugs in the Linux kernel. Sun et al. [53] employ LLMs to assist static analysis in identifying logic vulnerabilities in smart contracts. Li et al. [40] propose IRIS, which first leverages LLMs to identify the specifications of source/sinks for static analysis tools CodeQL for repository-level vulnerability detection. Wang et al. [56] propose InferROI, which employs LLMs to infer resource-oriented intentions in code, combined with a two-stage static analysis approach to inspect control-flow paths for potential resource leak detection.

2.1.2 Empirical Studies. Many efforts have been dedicated to evaluating LLMs in bug detection [22, 27, 29], covering diverse benchmarks, LLMs, and metrics. However, these benchmarks are primarily derived from synthetic datasets (e.g., Juliet [10]) and open-source projects, whose bug distributions may differ significantly from those encountered in real-world proprietary enterprise codebases. Furthermore, prior work has not systematically evaluated the ability of LLMs to reduce false positives in static bug detection. This study

fills this research gap by providing a comprehensive evaluation of LLMs in the context of false positive reduction in enterprise-level static bug detection.

2.2 False Positive Elimination in Bug Detection

2.2.1 Static Analysis-Based False Positive Elimination. A number of studies have explored the use of more precise static analysis techniques to identify and eliminate false positives from bug reports. These efforts primarily fall into two categories: model checking-based and symbolic execution-based approaches. Model checking-based approaches [46, 54] abstract the program into a finite-state model and use formal verification tools to exhaustively explore all possible execution states in order to determine the feasibility of reported paths. Symbolic execution-based approaches [26, 34] symbolically execute the program to collect path constraints and verify path feasibility using SMT solvers. Despite their precision, both categories face significant challenges. Exhaustively exploring all paths leads to path explosion, and their implementations are often tightly coupled with specific static analyzers or require handcrafted rules for different vulnerability types, resulting in poor generalization and restricting their application to large-scale systems.

2.2.2 Learning-Based False Positive Elimination. With the rise of deep learning, several approaches have been proposed to train predictive models that distinguish real bugs from false positives based on features extracted from source code or warning reports. Lee et al. [31] train a convolutional neural network (CNN) to learn lexical patterns from code segments that trigger alarms. Kharkar et al. [30] adopt a Transformer-based model to identify false positive warnings. However, these supervised learning-based approaches require large-scale labeled training data for each bug type, which limits their generalizability across different bug types or in an enterprise where training data is unavailable. Recent work combines LLMs and static analysis information to identify false positives during static bug detection. Wen et al. [59] propose LLM4SA, which provides LLMs with all the code contexts around the sink (e.g., callers and callees of the sink and including hundreds of lines of code) into one single prompt. Wang et al. [57] propose LLMDFA, which leverages few-shot chain-of-thought prompting to guide LLMs to analyze the path feasibility within each function. Du et al. [23] propose LLM4PFA, which integrates static analysis-derived path constraints with agent-driven, context-aware reasoning, enabling the model to more accurately assess source-to-sink reachability and thereby eliminate false positives caused by infeasible paths. Li et al. [35] propose BUGLENS, a post-refinement framework that guides LLMs through structured reasoning steps to assess security impact and validate constraints from the source code. This work is parallel to ours.

2.2.3 Empirical Studies. Guo et al. [28] provide a comprehensive review of various methods for mitigating false positive static analysis warnings, highlighting the key challenges in this field based on their survey results. Specifically, their study categorizes false positive elimination methods into five distinct approaches: Statistical Probability, Dynamic Program Testing, Machine learning, and Clustering. However, their study does not explore the potential of LLMs in mitigating false positives. Moreover, it focuses on false

positives from open-source static analyzers, offering limited insight into the effectiveness and challenges of addressing false positives in enterprise settings. In addition, several works conduct evaluations and empirical studies of static analysis tools. Li et al. [36] and Lipp et al. [41] compare and evaluate the effectiveness and performance of JAVA and C SASTs, respectively. Ami et al. [15] conduct in-depth interviews with 20 practitioners to investigate how developers perceive, select, and use SASTs. Li et al. [38] investigate why industrial professionals utilize SAST benchmarks, identify barriers to their usage, and explore potential improvements for existing benchmarks. However, these works do not provide a comprehensive study of SAST false positives and how to systematically eliminate them. Our work fills this gap.

3 Research Setting

The goal of our empirical study is to investigate the prevalence of false positives produced by static analysis tools in industrial codebases and to evaluate the effectiveness of existing learning-based approaches in mitigating these false positives. This section outlines the overall research setting. Section 3.1 details how we collected real-world false positive data from an internal static analyzer within Tencent. Section 3.2 introduces the research questions that guide this study and the specific settings for each research question.

3.1 Data Collection

We collected data from the main repositories of Tencent’s Advertising division, encompassing all projects within the Advertising and Marketing Services (AMS) business line. The dataset covers warnings reported between September 2024 and June 2025, including both real bugs and false positives. The overall data analysis and collection pipeline is summarized in Table 1. In particular, the process involves three key steps: **(1) Automated scanning.** A static analyzer was applied to the codebases to generate warnings. **(2) Manual review.** Enterprise code reviewers manually examined the warnings to determine whether they represented real bugs or false positives. **(3) Data collection.** We collected the verified outcomes from the manual review records, followed by a secondary validation to ensure correctness and data integrity.

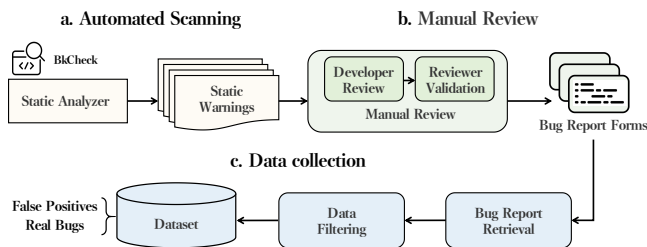


Figure 1: Overview of Data Collection

Static Analyzer. Mature open-source and commercial static analysis tools often fall short of meeting the comprehensive requirements of enterprises during code review, particularly with respect to efficiency, workflow integration, security, and cost. As a result, Tencent developed its own static analysis tool, BkCheck, to support large-scale static scanning ensuring both high efficiency and security.

BkCheck is a proprietary static analysis tool developed by Tencent’s CodeCC platform. It covers a wide range of detection rules, including null pointer dereference, logic defects, divide-by-zero errors, and memory leaks. Compared to compiler-based analyzers such as Coverity [6] and CodeQL [5], BkCheck does not require code compilation, which allows for faster scans and seamless integration into large-scale industrial projects. It has been widely deployed in numerous flagship Tencent products, including WeChat Pay and many popular Tencent games.

Manual Review. During the code review phase, both developers and professional code reviewers manually inspected the warnings reported by static analyzers. The review process consisted of two independent rounds conducted by different reviewers. In the first round, developers examined each warning together with the corresponding code to determine whether it was a false positive or a real bug. For warnings identified as false positives, developers were required to document the reason; for real bugs, the analyzer-reported root cause was retained. In the second round, dedicated code reviewers validated the results of the first round by re-examining the code, warnings, and the initial annotations. The final outcomes from both rounds were consolidated into specialized bug report forms, which recorded the code snippet, bug category, and final review decision. It is worth noting that this review phase is part of Tencent’s routine code review workflow. As such, the resulting manual annotations reflect the professional judgment of experienced enterprise developers and reviewers regarding false positives and real bugs. Consistent with the “security-first” principle, it is highly unlikely that true positives were inadvertently discarded. In cases where reviewers could not reach a definitive conclusion, the warning was marked as unresolved for follow-up verification by more senior reviewers.

Data Collection. We constructed our dataset from the finalized bug report forms generated during the above review phase. These forms contained 35 categories of bugs, as well as a portion of uncategorized cases. *Bug Types.* For this study, we selected the three most frequently occurring and developer-validated categories: Null Pointer Dereference (NPD), Out-of-Bounds (OOB), and Divide-by-Zero (DBZ). These categories accounted for approximately 52%, 24%, and 10% of all recorded warnings, respectively, covering 86% in total and highlighting their significant impact on enterprise-scale software systems. *Filtering Criteria.* We systematically extracted all records of these three categories between September 2024 and June 2025. Although our double checks revealed no correctness issues in the review forms, certain entries corresponded to deprecated code or contained incomplete contextual information, which could compromise the fairness of the evaluation. To ensure the rigor and reliability of the dataset, we applied the following filtering criteria: (1) Records associated with deprecated code were removed. (2) Records with incomplete code information or unavailable dependent context were excluded. (3) Cases with unresolved uncertainty regarding false positive status were discarded. (4) Duplicate records were eliminated. *Data Statistics.* After filtering, we obtained a total of 328 false positives and 105 real bugs. Table 1 summarizes the detailed statistics of the resulting dataset.

Table 1: Statistics of Dataset

Type	False Alarm Number	Real Bug Number	Total
NPD	107	29	136
DBZ	139	19	158
OOB	82	57	139
Total	328	105	433

3.2 Research Questions

Based on the enterprise-level dataset we collected in real-world static bug detection, we addressed the following research questions:

Research Question 1 (Prevalence of false positives in industry): What is the prevalence of false positives generated by static analysis tools in industrial practice?

To answer this question, we first perform interviews with Tencent developers to investigate the application practice of SATs, as well as analyzed the collected data. Specifically, we investigate (1) the industrial use of SATs in Tencent (i.e., how SATs are integrated into the code review workflow), (2) the false positive rates, and (3) the manual effort spent in validating reported false alarms.

Interview Protocol. We designed an online survey questionnaire containing open-ended questions to guide the interviews. The study protocol was approved by the Institutional Review Boards (IRBs) of the authors' institutions. The complete list of interview questions is provided in our replication package [11].

Interview Recruitment. We recruited five participants from Tencent's Advertising and Marketing Services (AMS) business line who were directly involved in software development and code review. The group consisted of two developers and three code reviewers, each with more than five years of professional experience.

Research Question 2 (Effectiveness of LLMs in reducing false alarms): How effective are a diverse range of LLM-based techniques in reducing false alarms?

With the rapid advancement of LLMs, they have demonstrated promising performance across a wide range of code-related tasks. Their vulnerability detection capabilities have also been validated on synthetic datasets (e.g., Juliet [10]) and relatively simple open-source function-level datasets (e.g., Devign [68]). This research question first investigates the practical effectiveness of LLMs in reducing false positives generated by SATs on real-world industrial data. Specifically, we address the following four questions:

- **RQ2.a.** What is the effectiveness of directly applying different LLMs?
- **RQ2.b.** What is the effectiveness of employing advanced prompting strategies with LLMs?
- **RQ2.c.** What is the effectiveness of advanced LLM-based methods combined with static analysis in reducing false positives?
- **RQ2.d.** How do LLM-based techniques compare with traditional learning-based techniques in reducing false positives?

Model selection. To evaluate the impact of different backbone LLMs, we selected four representative state-of-the-art models that

are widely adopted in bug detection tasks. These include two closed-source models, GPT-4o [3] and Claude-Opus-4 [1], as well as two open-source models, Qwen-3-Coder [4] and DeepSeek-R1 [2].

Prompting strategies. Beyond a basic prompt setting, we adopt three most widely used and representative prompting strategies following previous research [42, 47, 65].

- **Chain-of-Thought (CoT) strategy** [58, 64] explicitly guides LLMs to perform step-by-step reasoning, thereby improving their ability to handle complex logical dependencies when assessing whether a warning is a false positive.
- **Few-shot learning strategy** [17] provides LLMs with representative examples of both false positives and real bugs. By observing these demonstrations, LLMs learn decision boundaries and make more accurate, expert-like classifications.
- **Bug-type augmentation** [67] enriches prompts with structured bug types description information as bug knowledge to LLMs. This strategy supplies LLMs with explicit semantic cues about common bug patterns, enabling more informed reasoning over static analysis warnings.

Both few-shot learning and bug-type augmentation can be regarded as forms of domain knowledge enhancement. Bug-type augmentation provides relatively general domain knowledge, while few-shot learning offers concrete representative examples that serve as domain-specific guidance.

Hybrid methods between LLM and static analysis. Recent studies [23, 59] have proposed LLM-based approaches that explicitly integrate static analysis information for false positive reduction. However, these works have only been evaluated on open-source datasets, and their effectiveness in real-world industrial settings remains uncertain. To bridge this gap, we evaluate the following methods on our enterprise dataset:

- **LLM4SA** [59] leverages LLMs to directly analyze bug warning reports generated by static analysis tools. It supplies the model with extensive code context surrounding the reported sink (e.g., callers, callees, and hundreds of lines of relevant code) and prompts the LLM to determine whether the warning constitutes a false positive.
- **LLM4PFA** [23] improves complex inter-procedural path feasibility analysis to reduce false positives reported by static bug analyzers. Specifically, LLM4PFA integrates static analysis-derived path constraints with agent-driven, context-aware reasoning, enabling the model to more accurately assess source-to-sink reachability and thereby eliminate false positives caused by infeasible paths.

Although several other hybrid methods could also be used for false positive reduction, they cannot be directly applied to our target codebase for two reasons: (1) Tool dependency. For example, IRIS [40] relies on CodeQL for core taint analysis; however, CodeQL requires a fully compilable repository, which limits its generalizability. (2) Bug-type constraints. For instance, several bug types in our dataset fall outside the types supported by LLM4PFA [57], preventing a fair application. Therefore, we select LLM4SA and LLM4PFA as our baselines, as they are representative and pose fewer applicability constraints. Due to space limitations, we do not include the detailed prompts here. All implementation code for the baselines is available in our replication package [11].

Compare with traditional learning-based techniques. To investigate this, we first surveyed review studies evaluating learning-based models for vulnerability detection [21, 45, 51] and selected four representative state-of-the-art methods. These include two graph neural network (GNN)-based approaches and two methods that fine-tune pre-trained language models (PLMs):

GNN-based methods:

- **Devign [68]:** A GNN-based vulnerability detection model that constructs a composite graph representation of source code by integrating multiple program structures, including the abstract syntax tree (AST), control-flow graph (CFG), and data-flow graph (DFG), then applies GNNs to learn semantic features from this graph representation for vulnerability prediction.
- **ReGVD [44]:** A GNN-based model combines two GNN variants, ReGGNN and ReGCN, to enhance graph representation learning, thereby improving vulnerability detection accuracy.

PLM-based methods:

- **LLMAO [63]:** A PLM-based fault localization approach that fine-tunes CodeGen on fault localization and vulnerability detection datasets to improve detection precision.
- **LineVul [25]:** A PLM-based vulnerability detection model that fine-tunes CodeBERT, offering both function-level and line-level detection capabilities.

Research Question 3 (Costs Analysis): What are the costs of using LLMs in false positives reduction?

This RQ examines the practical costs of deploying LLM-based approaches for false positive reduction in enterprise settings. Although LLMs demonstrate strong potential for mitigating false positives, their adoption in industrial settings must also account for associated costs. We focus on two dimensions of cost:

- **Response time costs:** The average response time per case for different LLMs and methods.
- **Economic costs:** The monetary expense associated with API usage, measured as the average cost per case across models and methods.

By analyzing these factors, we aim to provide a comprehensive assessment of the feasibility and sustainability of adopting LLMs for false positive reduction in enterprise environments.

Research Question 4 (Breakdown and case analysis): How do LLMs perform in reducing false positives across different vulnerability categories, and what are their limitations?

Breakdown Analysis. False positives generated by static analysis tools may differ significantly across vulnerability categories, as each category involves distinct bug patterns and detection challenges. To better understand these variations, we investigate the performance of LLMs in reducing false positives across the three most frequent and impactful categories in our dataset: Null Pointer Dereference (NPD), Divide-by-Zero (DBZ), and Out-of-Bounds (OOB). This research question aims to reveal whether LLMs exhibit consistent effectiveness across categories.

Case Analysis. We perform bad case analysis to better understand the limitations of LLMs in identifying false positives. This analysis provides deeper insights into two dimensions: (i) *Error Patterns*. We categorize the representative bad cases where LLMs misdistinguish

between false positives and real bugs, and further analyze the conditions under which the LLMs' reasoning capabilities fall short. (ii) *Root Causes*. We further investigate the underlying reasons behind these bad cases, thereby providing a systematic understanding of the inherent limitations of LLMs in static bug detection.

4 Result and Analysis

4.1 Metrics and Setup

For effectiveness evaluation (i.e., RQ2-RQ4), we use four commonly-adopted metrics in bug detection tasks: accuracy, precision, recall, and F1. Additionally, we use two metrics introduced in LLM4PFA [23] to evaluate the effectiveness in identifying false positives generated by static analyzers: False Positive Reduction Precision (FPR_P) and False Positive Reduction Recall (FPR_R), which specifically measure the precision and recall for negative samples (i.e., false positives) in the dataset.

$$\text{FPR_P} = \frac{\text{TN}}{\text{TN} + \text{FN}} \quad \text{FPR_R} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

To mitigate potential concerns regarding LLM randomness, we executed each query three times per case and adopted a majority-vote strategy to determine the final outcome. For traditional deep-learning based methods, we directly applied the models trained on the original datasets used in their papers to conduct experiments, since no enterprise-level training dataset is publicly available.

4.2 RQ1: Prevalence of False Positives

We conducted interviews with five participants and analyzed their responses to the survey questionnaire. Based on this analysis, we derived the following interview results:

Industrial Practice of SATs. As illustrated in Figure 2, BkCheck, Tencent's enterprise-customized SAT, functions as a critical gated check-in by scanning each commit. The reported bug alarms undergo up to two rounds of manual inspection: first, developers manually review all reported alarms, and then those identified as false alarms are subjected to a second-round validation by a different reviewer to minimize the risk of overlooking potential bugs. Overall, SATs play a fundamental role in industrial software quality assurance, with their reported alarms being taken seriously as part of the enterprise code review process.

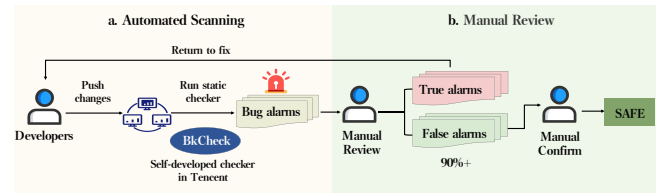


Figure 2: Overview of Code Review in Tencent

High False Positive Rate. In our final dataset from Tencent, the proportion of false positives was at 76% (e.g., 328 false positives and 105 true bugs). In fact, the real-world distribution of false positives could be even higher as a substantial number of false-positive records are excluded from our final dataset due to incomplete or inaccessible code contexts. When accounting for these discarded cases, the false positive rate of static warnings is higher than 90%.

Such a high false positive rate stems from a design trade-off: enterprise static analysis tools prioritize recall over precision to ensure that no potential bugs are overlooked. Consequently, false positive rates in enterprise settings are even higher than those of many open-source analyzers.

High Manual Effort for Validation. As shown in Figure 2, each reported bug alarm undergo up to two rounds of manual inspection to minimize the risk of overlooking potential bug. Based on our interviews, the average time required to examine a single alarm was approximately 10 minutes per round. As a result, one single bug alarm would cost 10 to 20 minutes of manual inspection effort. As hundreds of alarms can be reported by SATs for scanning large-scale enterprise software, significant human effort are wasted for inspecting false alarms in practice.

Answer to RQ1: SATs play a fundamental role in industrial software quality assurance, with their reported alarms being taken seriously as part of the enterprise code review process. However, false alarms are extremely prevalent (i.e., with 90% false alarm rate). Meanwhile, substantial manual effort are wasted to inspect reported bug alarms, with one false alarm on average taking 10 to 20 minutes of manual inspection. Consequently, the automatic reduction of false positives in SATs is a critical and urgent need in enterprise code review processes.

4.3 RQ2: Effectiveness of LLMs in Reducing False Alarms

To investigate the effectiveness of LLMs in reducing false positives, we conduct a comprehensive evaluation of various LLM-based false alarm reduction techniques on our Tencent datasets. Moreover, we compare the performance of LLM-based approaches with that of traditional learning-based methods.

4.3.1 Effectiveness of various LLM-based false alarm reduction techniques. Table 2 compares the performance of all LLM-based false alarm reduction techniques on our Tencent dataset using various backbone LLMs.

Overall Effectiveness. LLMs demonstrate strong potential for static false positive reduction, with hybrid methods that combine LLMs and static analysis (i.e., LLM4SA and LLM4PFA) achieving the highest accuracy of 0.93-0.94. These LLM-based approaches also provide a degree of interpretability, making their outputs more available to subsequent human validation.

Method Comparison. Across all evaluated strategies, methods combining LLMs with static analysis (i.e., LLM4SA and LLM4PFA) outperform standalone LLMs with different prompting strategies, which in turn surpass basic prompt-only methods. Among prompting strategies, Chain-of-Thought (CoT) delivers the weakest performance, even underperforming basic prompts in some cases. This may be due to these advanced LLMs have already internalized reasoning capabilities during the training process, and explicitly enforcing CoT prompts can paradoxically degrade performance. In contrast, few-shot enhanced prompting consistently yields the best results among the three prompt strategies, underscoring the value of providing concrete examples to guide LLM reasoning. Both LLM4SA and LLM4PFA not only demonstrate strong effectiveness, with all backbone LLMs achieving accuracy rates above 0.86, but

also exhibit greater stability across different backbone LLMs compared to methods relying solely on prompt strategies. These results further validate the potential of integrating LLMs with static analysis for effective false positive reduction.

Generalization Across LLMs. Overall, the four state-of-the-art LLMs exhibit comparable levels of effectiveness, though their relative strengths vary across methods and prompting strategies. For instance, Claude performs competitively with bug-type-enhanced and few-shot-enhanced prompts, but its performance is noticeably weaker with basic prompts. In contrast, Qwen-3-Coder delivers the most consistent performance across all prompting strategies. Importantly, LLM4PFA exhibits strong generalization across all models, achieving accuracy above 0.93 in every case. Given that enterprises typically prioritize recall over precision, LLM4PFA with Claude (recall = 0.88) and DeepSeek-R1 (recall = 0.86) are more suitable for industrial practice.

4.3.2 Compare with traditional learning-based techniques. Table 3 presents the results of four representative deep learning-based vulnerability detection methods on our Tencent dataset.

Overall Effectiveness. Overall, all evaluated LLM-based methods consistently outperform deep learning approaches. The latter exhibited markedly poor performance, with the highest accuracy reaching only from 0.36 to 0.44 (except for LineVul). Two key factors may account for this limitation. First, deep learning-based methods are highly dependent on training data distribution. These learning-based models trained on existing benchmark datasets perform poorly when directly applied to previously unseen enterprise code, where large-scale, labeled training data is scarce. This lack of sufficient domain-specific training data makes fine-tuning such models for enterprise use particularly challenging. Second, prior studies [24] have shown that learning-based approaches often struggle to capture the true root causes of bugs. Instead, they tend to rely on superficial statistical patterns or secondary code features correlated with buggy functions. As a result, these methods are inherently limited in their ability to generalize to real-world enterprise bug detection.

Methods Comparison. Although Linevul reports a seemingly high accuracy of 0.76, this result stems from its tendency to classify all warnings as false positives. Since our dataset contains a large proportion of false positive cases, this bias inflates its accuracy but undermines practical value. A likely reason is that LineVul was trained on the highly imbalanced BigVul dataset, where most samples are non-vulnerable. By contrast, the other methods all achieve accuracies below 0.44. Among them, ReGVD stands out due to its comparatively high recall of 0.87. However, its false positive recall is only 0.19, meaning that it only successfully eliminates 19% of false positives. This limited effectiveness in reducing false alarms still imposes substantial manual validation costs, restricting its overall practical benefit.

Interpretability. Traditional learning-based methods inherently suffer from poor interpretability. Their outputs are limited to predicted labels without any explanatory context, which increases the burden of manual validation during enterprise code review. In contrast, LLM-based methods provide natural language explanations alongside predictions. These explanations significantly support human reviewers.

Table 2: Effectiveness of LLM-based Methods with different LLMs

Technique	Model	Accuracy	Precision	Recall	F1	FPR_P	FPR_R
Basic Prompt	GPT-4o	0.50	0.28	0.66	0.39	0.80	0.45
	Claude-Opus-4	0.40	0.26	0.83	0.40	0.83	0.26
	Qwen-3-Coder	0.55	0.29	0.62	0.40	0.81	0.52
	DeepSeek-R1	0.43	0.27	0.84	0.41	0.85	0.29
Cot Enhanced	GPT-4o	0.49	0.29	0.76	0.42	0.84	0.40
	Claude-Opus-4	0.42	0.29	0.94	0.44	0.93	0.25
	Qwen-3-Coder	0.48	0.29	0.79	0.42	0.85	0.38
	DeepSeek-R1	0.40	0.28	0.90	0.42	0.89	0.24
Few-shot Enhanced	GPT-4o	0.50	0.26	0.64	0.37	0.79	0.43
	Claude-Opus-4	0.72	0.46	0.74	0.57	0.90	0.72
	Qwen-3-Coder	0.71	0.40	0.59	0.48	0.85	0.72
	DeepSeek-R1	0.69	0.43	0.83	0.56	0.92	0.65
Bug Type Enhanced	GPT-4o	0.53	0.27	0.54	0.36	0.78	0.52
	Claude-Opus-4	0.67	0.41	0.87	0.56	0.93	0.60
	Qwen-3-Coder	0.65	0.37	0.64	0.47	0.85	0.65
	DeepSeek-R1	0.64	0.38	0.83	0.52	0.91	0.57
LLM4SA	GPT-4o	0.86	0.63	0.84	0.72	0.94	0.84
	Claude-Opus-4	0.89	0.68	0.72	0.70	0.91	0.89
	Qwen-3-Coder	0.92	0.85	0.65	0.74	0.90	0.96
	DeepSeek-R1	0.91	0.77	0.86	0.81	0.95	0.92
LLM4PFA	GPT-4o	0.93	0.93	0.75	0.83	0.93	0.98
	Claude-Opus-4	0.93	0.83	0.88	0.85	0.96	0.94
	Qwen-3-Coder	0.94	0.93	0.79	0.86	0.94	0.98
	DeepSeek-R1	0.93	0.84	0.86	0.85	0.95	0.95

Table 3: Effectiveness of traditional learning-based methods

Tech.	Acc.	Pre.	Recall	F1	FPR_P	FPR_R
Devign	0.44	0.35	0.61	0.44	0.60	0.34
ReGVD	0.36	0.26	0.87	0.39	0.82	0.19
LLMAO	0.36	0.22	0.67	0.34	0.71	0.26
LineVul	0.76	0.00	0.00	0.00	0.76	1.00

Answer to RQ2: LLMs demonstrate strong potential for reducing the false positives reported by SATs in industrial settings. Among all LLM-based approaches, hybrid techniques that integrate LLMs with static analysis (i.e., LLM4PFA) achieve the best effectiveness, successfully eliminating 94–98% of false positives across different backbone LLMs while maintaining high recall. In contrast, traditional learning-based vulnerability detection methods perform poorly due to the scarcity of large-scale, high-quality enterprise training data. Furthermore, LLM-based methods offer superior interpretability by augmenting label-only predictions with natural language explanations, thereby increasing the practicality and usability of their outputs for subsequent manual validation.

4.4 RQ3: Costs Analysis

In order to provide a comprehensive assessment of the feasibility and sustainability of adopting LLMs for false positive reduction in enterprise environments, Table 4 compares the time costs and economic costs of different LLM-based methods using various backbone LLMs.

Overall, LLM-based methods fall well within the acceptable range of both time and economic costs for an enterprise environment. **Time Cost Analysis.** The average processing cost per alarm ranges from 2.1 to 109.5 seconds, depending on the specific method and backbone model. Even the most time-consuming LLM, DeepSeek-R1 (i.e., about 1–2 minutes per warning), remains substantially lower than manual validation, which requires approximately 20 minutes

Table 4: Cost of LLM-based Methods with different LLMs

Tech.	Model	Ave. time cost (s)	Ave. API cost (\$)
Basic Prompt	GPT-4o	5.9	0.015
	Claude-Opus-4	12.8	0.065
	Qwen-3-Coder	2.1	0.0013
	DeepSeek-R1	109.5	0.0046
Few-shot Prompt	GPT-4o	7.9	0.020
	Claude-Opus-4	14.8	0.120
	Qwen-3-Coder	8.1	0.0011
	DeepSeek-R1	60.5	0.0018
LLM4SA	GPT-4o	8.6	0.017
	Claude-Opus-4	15.2	0.094
	Qwen-3-Coder	17.2	0.0084
	DeepSeek-R1	60.2	0.0024
LLM4PFA	GPT-4o	12.6	0.020
	Claude-Opus-4	16.7	0.120
	Qwen-3-Coder	16.1	0.0056
	DeepSeek-R1	58.3	0.0025

per warning across multiple review rounds. Moreover, when integrated with DeepSeek, LLM4PFA achieves an accuracy of 0.93 while maintaining a recall of 0.86, making it highly effective for assisting enterprises in false positive reduction, as it can replace at least one round of manual review and save considerable human effort without compromising reliability.

Economic Cost Analysis. The average expense per alarm is 0.0011\$ – 0.12\$. In addition, two open-source models can be deployed within enterprise infrastructure, eliminating API-related costs. Even the most expensive commercial model (i.e., Claude-Opus-4, 0.12\$ per warning) remains negligible compared to the substantial savings in manual effort, underscoring the usability of LLM-based approaches.

Answer to RQ3: LLM-based false alarm reduction techniques are cost-effective in industrial settings, incurring both acceptable time and economic costs. On average, the processing time per alarm ranges from 2.1 to 109.5 seconds, while the expense per alarm remains as low as \$0.0011–\$0.12. Compared to the 10–20 minutes typically required for manual inspection, these

techniques offer a substantial reduction in developer effort, underscoring their practical value for mitigating false positives in static analysis within industry.

4.5 RQ4: Breakdown and Case Analysis

Breakdown Analysis. Figure 3 illustrates the effectiveness variety of LLM-based false alarm reduction techniques across different bug categories. Overall, the effectiveness of different LLM-based methods varies across bug categories. Almost all methods achieve their best performance on Divide-by-Zero (DBZ) bugs, largely because the bug patterns in this category are relatively simple and consistent, making them easier for LLMs to identify. In contrast, Null Pointer Dereference (NPD) bugs exhibit more complex and diverse patterns. Moreover, many NPD false positive cases involve intricate conditional constraints, which pose significant challenges for LLMs to reason accurately about program behavior, resulting in comparatively lower performance in this category.

Answer to RQ4 (Breakdown Analysis): The effectiveness of LLM-based methods vary across different bug categories. In Tencent datasets, LLM-based methods achieve the best performance on Divide-by-Zero (DBZ) bugs, while their performance is lowest on Null Pointer Dereference (NPD) bugs.

Case Analysis. To better understand the limitations of LLMs in reducing false positives, we conducted a manual analysis of two types of failure cases: (1) cases where different prompt strategies failed to help LLMs distinguish between false positives and real bugs, and (2) cases where all methods failed to classify correctly. Although confidentiality constraints prevent us from presenting the actual code examples, our analysis reveals several key limitations of LLMs in bug detection.

(i) *Limited capacity for long-context reasoning.* We observed that LLMs are significantly less effective at mitigating false positives in cases involving long code contexts. For cases where all methods failed, the average length of the function containing the warning was 95.6 lines longer than the overall dataset average, and 67.7 lines longer than cases only correctly classified by LLM4PFA but missed by other LLM-based methods. This demonstrates that LLMs struggle with extended context reasoning. Although LLM4PFA partially alleviates this issue, the limitation persists.

(ii) *Difficulty with complex cascaded constraints.* When analyzing code containing numerous loops, deeply nested conditional branches, or intricate logical expressions, LLMs often overlook boundary conditions or introduce logical inconsistencies. As a result, they fail to accurately track variable states across multiple reasoning steps. For the cases misclassified by all methods, the average number of conditional statements was 21 higher than the overall dataset average, and 18.9 higher than the subset solvable by LLM4PFA alone. This gap highlights the intrinsic weakness of LLMs in handling multi-step, complex cascaded constraints reasoning.

(iii) *Insufficient semantic understanding.* LLMs also struggle with less common syntactic constructs, pointer manipulations, and user-defined, deeply nested data structures. Their training often lacks sufficient exposure to such patterns, leading to misinterpretation and erroneous reasoning when analyzing these cases.

Answer to RQ4 (Case Analysis): Although LLMs demonstrate promise in reducing false positives, they still face fundamental limitations in enterprise-scale bug detection. Specifically, LLMs exhibit (1) limited effectiveness in handling long code contexts, (2) difficulty reasoning complex cascaded constraints, and (3) insufficient semantic understanding. These challenges remain critical directions for advancing the practical effectiveness of LLMs in future work.

5 Discussions

5.1 Implications for Practitioners

5.1.1 How LLMs Help in Practice. Our results from RQ2-RQ4 demonstrate that several mature LLM-based approaches (i.e., LLM4SA and LLM4PFA) can effectively assist code reviewers in reducing static false positives in real-world enterprise settings. Compared with manually crafted rules, which suffer from poor scalability and limited coverage, and traditional deep learning approaches, which struggle to generalize to proprietary data and often lack interpretability, LLM-based methods offer a more balanced trade-off between effectiveness, interpretability, and efficiency.

Effectiveness. The best-performing LLM-based method LLM4PFA achieved an accuracy of 0.93–0.94 across different state-of-the-art backbone models on our dataset, demonstrating strong potential for false positive reduction. However, recall remains below the enterprise-expected threshold of 90%, indicating that some degree of manual review is still necessary to guarantee absolute safety. Nevertheless, LLM-based methods can feasibly replace at least one round of manual review in multi-stage validation workflows, reducing overall human effort significantly.

Despite their promise, current LLM-based approaches face several limitations: (1) Long-context reasoning. LLMs still struggle to analyze warnings requiring deep or cross-module context [16, 37]. The strong performance observed in our dataset partly reflects the limited context sensitivity of the static analyzers used, where most warnings involve only short code fragments. For cases requiring deeper interprocedural reasoning, effectiveness degrades. (2) Complex constraint analysis. LLMs have difficulty accurately simulating program behavior in cases involving cascading constraints [23]. While LLM4PFA alleviates this to some extent, the challenge has not been fully resolved. (3) Bug type coverage. Both prior work and this study focus mainly on memory safety bugs, which exhibit clearer patterns. The performance of LLMs on less common bug types remains an open question. (4) Randomness in reasoning. LLM outputs exhibit randomness. For more stable results, majority voting across multiple runs (e.g., three per case in our evaluation) is recommended. Moreover, while LLMs show strong potential in filtering false positives, their capacity to discover new bugs remains bounded by the detection capabilities of static analyzers themselves. How to leverage LLMs to help enterprises identify new bugs beyond the coverage of static analyzers remains an open challenge.

Interpretability. Unlike most traditional deep learning models that output only binary labels, LLM-based methods provide natural language explanations alongside predictions. These explanations significantly support human reviewers, reducing the average time to validate a warning from about 10 minutes to approximately 3

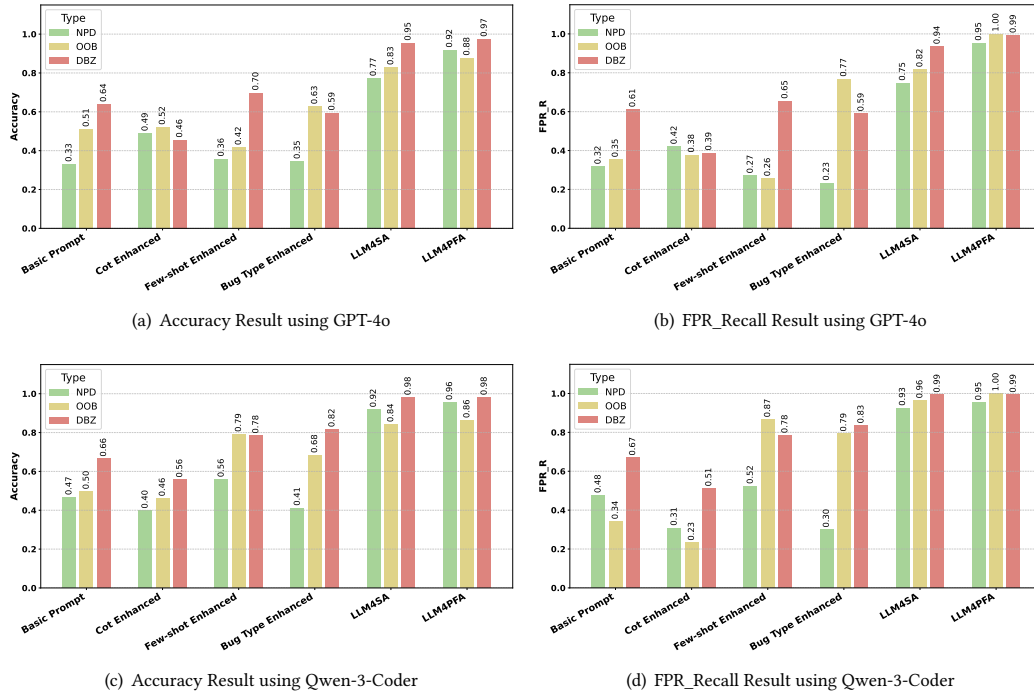


Figure 3: Comparison of performance for LLM-based methods

minutes. Even when explanations contain inaccuracies, they still aid in quicker issue identification.

Efficiency. As shown in RQ3, all evaluated LLMs perform static false positive reduction at much lower time and acceptable economic costs compared to manual review. Given that model performance is broadly comparable across LLMs, model selection in practice may emphasize efficiency. As such, the open-source Qwen-3-Coder achieves a particularly favorable balance between time and economic cost, while also avoiding the practical challenges of deploying closed-source models in enterprise environments.

5.2 Implications for Researchers

From a research perspective, our findings highlight several directions for improving the use of LLMs in static bug detection.

Knowledge-enhanced LLMs. RQ2 shows that incorporating domain knowledge significantly improves LLMs reasoning accuracy. Both bug-type descriptions and illustrative examples proved beneficial, with examples yielding greater gains by explicitly exposing recurring bug patterns. In this study, such examples were manually constructed to guide LLM reasoning. Future research could explore automated approaches for acquiring targeted knowledge (e.g., RAG for high-quality historical cases), as well as investigating diverse knowledge sources and modalities for enriching LLMs. These directions may enhance not only effectiveness but also interpretability.

Complex contextual and cascaded constraints analysis. Currently, LLMs demonstrate strong capabilities in analyzing simple code snippets but struggle with scenarios requiring complex context and cascaded constraints reasoning, which are prevalent in real-world software development. Promising approaches to enhancing LLMs in this direction include decomposing intricate reasoning

tasks into smaller, more manageable subproblems where LLMs perform more reliably, or designing hybrid pipelines to support complex contextual analysis.

Combining LLMs with Static Analysis. LLMs and static analysis complement each other effectively. Specifically, LLMs excel at understanding code snippets and generalizing across patterns, so they can enrich static analysis by providing scalable domain knowledge and expanding the coverage of analysis rules. However, LLMs face limitations such as hallucinations and difficulties with long-context reasoning tasks. Static analysis can mitigate these issues by providing reliable program facts and intermediate results, which help reduce hallucinations and assist LLMs in breaking down complex reasoning tasks. The competitive results of LLM4PFA further demonstrate the value of deeply integrating LLMs with static analysis. Thus, combining LLMs with static analysis holds great promise as a future direction for static bug detection.

6 Threats to Validity

Construct Validity. A potential threat arises from the reliability of manual labeling. To mitigate this, all labels were reviewed through multiple rounds by experienced in-house developers and professional code reviewers, providing strong domain expertise and consistency. Our research team performed further validation, and did not identify any correctness issues in the reviewed labels.

Internal Validity. Due to the randomness of LLMs, there is an inherent risk to the reliability and consistency of experimental results. To mitigate this, we conducted multiple trials for each LLM-based method and reported the majority outcomes across repeated runs. Moreover, the results of all methods with different LLMs are included in our replication package [11] to ensure transparency and

reproducibility. Another threat is the limited dataset scope. Due to enterprise constraints, we collected all available data and applied rigorous filtering to ensure quality. The final dataset includes 433 real cases spanning 10 months, which we believe remains valuable for identifying common false-positive patterns and evaluating the effectiveness of LLMs in false positive reduction.

External Validity. Our data is collected from Tencent and primarily consists of C/C++ programs, which may limit generalizability to other enterprise environments or languages. Nevertheless, the industrial scale of the dataset still provides meaningful insights for future work. Since the evaluated methods are language-agnostic, they can be extended to other programming languages. In future work, we plan to expand the dataset to include additional business lines and programming languages to further enhance the generalizability of our results.

7 Conclusion

This work conducts the first comprehensive empirical study of diverse LLM-based false alarm reduction techniques in an industrial context at Tencent, one of the largest IT companies in China. Through interviewing developers and analyzing the data, our results highlight the prevalence and bad impact of false positives. Meanwhile, our results show the huge potential of LLMs for reducing false alarms in industrial settings (with best techniques eliminating 94–98% of false positives). Furthermore, LLM-based techniques are cost-effectiveness, with low time and money costs. Based on findings, we further discuss the practical implications for practitioners and outline promising directions for future research.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62332005).

References

- [1] 2024. *Claude Sonnet 3.5*. <https://www.anthropic.com/news/claude-3-family>
- [2] 2024. *DeepSeek-Coder-V2-Instruct*. <https://huggingface.co/deepseek-ai/DeepSeek-Coder-V2-Instruct>
- [3] 2024. *GPT-4-turbo*. <https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo>
- [4] 2024. *Qwen2.5-Coder-32B-256 Instruct*. <https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct>
- [5] 2025. *CodeQL*. <https://codeql.github.com/>
- [6] 2025. *Coverity*. <https://scan.coverity.com>
- [7] 2025. *CppCheck*. <http://cppcheck.net/>
- [8] 2025. *Gawk*. <https://github.com/gvxl/gawk>
- [9] 2025. *Infer*. <https://fbinfer.com/>
- [10] 2025. *Juliet Test Suite C/C++*. <https://samate.nist.gov/SARD/test-suites/112>
- [11] 2025. *Replication package*. <https://github.com/LLM4SFPR/static-bug-false-positives-reduction-in-practice>
- [12] 2025. *Tencent*. <https://www.tencent.com>
- [13] 2025. *Tencent Cloud*. <https://cloud.tencent.com>
- [14] 2025. *WeChat*. <https://weixin.qq.com>
- [15] Amit Seal Ami, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. 2024. “False negative - that one is going to kill you”: Understanding Industry Perspectives of Static Analysis based Security Testing. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3979–3997. <https://doi.org/10.1109/sp54263.2024.00019>
- [16] Shengnan An, Zexiong Ma, Zeqi Lin, Nanning Zheng, Jian-Guang Lou, and Weizhu Chen. 2024. Make your llm fully utilize the context. *Advances in Neural Information Processing Systems* 37 (2024), 62160–62188.
- [17] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [18] Yuandao Cai, Peisen Yao, Chengfeng Ye, and Charles Zhang. 2023. Place your locks well: understanding and detecting lock misuse bugs. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3727–3744.
- [19] Yuandao Cai, Peisen Yao, and Charles Zhang. 2021. Canary: practical static detection of inter-thread value-flow bugs. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 1126–1140.
- [20] Yuandao Cai, Chengfeng Ye, Qingkai Shi, and Charles Zhang. 2022. Peahen: Fast and precise static deadlock detection via context reduction. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 784–796.
- [21] Partha Chakraborty, Krishna Kanth Arumugam, Mahmoud Alfadhel, Meiyapan Nagappan, and Shane McIntosh. 2024. Revisiting the performance of deep learning-based vulnerability detection on realistic datasets. *IEEE Transactions on Software Engineering* 50, 8 (2024), 2163–2177.
- [22] Yangruibo Ding, Yanjun Fu, Omniyiah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2024. Vulnerability detection with code language models: How far are we? *arXiv preprint arXiv:2403.18624* (2024).
- [23] Xueying Du, Kai Yu, Chong Wang, Yi Zou, Wentai Deng, Zuoyu Ou, Xin Peng, Lingming Zhang, and Yiling Lou. 2025. Minimizing False Positives in Static Bug Detection via LLM-Enhanced Path Feasibility Analysis. *arXiv preprint arXiv:2506.10322* (2025).
- [24] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. *arXiv preprint arXiv:2406.11147* (2024).
- [25] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *19th IEEE/ACM International Conference on Mining Software Repositories, MSR 2022, Pittsburgh, PA, USA, May 23–24, 2022*. ACM, 608–620. <https://doi.org/10.1145/3524842.3528452>
- [26] Mikhail R Gadelha, Enrico Steffnlongo, Lucas C Cordeiro, Bernd Fischer, and Denis Nicole. 2019. Smt-based refutation of spurious bug reports in the clang static analyzer. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 11–14.
- [27] Zeyu Gao, Hao Wang, Yuchen Zhou, Wenyu Zhu, and Chao Zhang. 2023. How Far Have We Gone in Vulnerability Detection Using Large Language Models. *CoRR abs/2311.12420* (2023). <https://doi.org/10.48550/ARXIV.2311.12420>
- [28] Zhaoqiang Guo, Tingting Tan, Shiran Liu, Xutong Liu, Wei Lai, Yibiao Yang, Yanhui Li, Lin Chen, Wei Dong, and Yuming Zhou. 2023. Mitigating false positive static analysis warnings: Progress, challenges, and opportunities. *IEEE Transactions on Software Engineering* 49, 12 (2023), 5154–5188.
- [29] Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, and Mayur Naik. 2023. Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities. *CoRR abs/2311.16169* (2023). <https://doi.org/10.48550/ARXIV.2311.16169>
- [30] Anant Kharkar, Roshanak Zilouchian Moghaddam, Matthew Jin, Xiaoyu Liu, Xin Shi, Colin Clement, and Neel Sundaresan. 2022. Learning to reduce false positives in analytic bug detectors. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1307–1316. <https://doi.org/10.1145/3510003.3510153>
- [31] Seongmin Lee, Shin Hong, Jungbae Yi, Taeksu Kim, Chul-Joo Kim, and Shin Yoo. 2019. Classifying false positive static checker alarms in continuous integration using convolutional neural networks. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 391–401.
- [32] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2023. The Hitchhiker’s Guide to Program Analysis: A Journey with Large Language Models. (2023). *arXiv:2308.00245* [cs.SE]
- [33] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2024. Enhancing Static Analysis for Practical Bug Detection: An LLM-Integrated Approach. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 111 (apr 2024), 26 pages. <https://doi.org/10.1145/3649828>
- [34] Hongzhe Li, Taebeom Kim, Munkhbayar Bat-Erdene, and Heejo Lee. 2013. Software vulnerability detection using backward trace analysis and symbolic execution. In *2013 International Conference on Availability, Reliability and Security*. IEEE, 446–454.
- [35] Haonan Li, Hang Zhang, Kexin Pei, and Zhiyun Qian. 2025. Towards More Accurate Static Analysis for Taint-Style Bug Detection in Linux Kernel. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025*.
- [36] Kaixuan Li, Sen Chen, Lingling Fan, Ruitao Feng, Han Liu, Chengwei Liu, Yang Liu, and Yixiang Chen. 2023. Comparison and evaluation on static application security testing (sast) tools for java. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 921–933.
- [37] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. 2024. Long-context llms struggle with long in-context learning. *arXiv preprint*

- arXiv:2404.02060 (2024).
- [38] Yuan Li, Peisen Yao, Kan Yu, Chengpeng Wang, Yaoyang Ye, Song Li, Meng Luo, Yepang Liu, and Kui Ren. 2025. Understanding Industry Perspectives of Static Application Security Testing (SAST) Evaluation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 3033–3056.
 - [39] Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238 [cs.CR] <https://arxiv.org/abs/2405.17238>
 - [40] Ziyang Li, Saikat Dutta, and Mayur Naik. 2025. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238 [cs.CR] <https://arxiv.org/abs/2405.17238>
 - [41] Stephan Lipp, Sebastian Banescu, and Alexander Pretschner. 2022. An empirical study on the effectiveness of static C code analyzers for vulnerability detection. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 544–555.
 - [42] Zhihong Liu, Zezhou Yang, and Qing Liao. 2024. Exploration on prompting LLM with code-specific information for vulnerability detection. In *2024 IEEE International Conference on Software Services Engineering (SSE)*. IEEE, 273–281.
 - [43] Qiheng Mao, Zhenhao Li, Xing Hu, Kui Liu, Xin Xia, and Jianling Sun. 2024. Towards Effectively Detecting and Explaining Vulnerabilities Using Large Language Models. arXiv:2406.09701 [cs.SE] <https://arxiv.org/abs/2406.09701>
 - [44] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. 2022. Regvd: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 178–182.
 - [45] Chao Ni, Xin Yin, Liyu Shen, and Shaohua Wang. 2026. Learning-based models for vulnerability detection: An extensive study. *Empirical Software Engineering* 31, 1 (2026), 18.
 - [46] Hendrik Post, Carsten Sinz, Alexander Kaiser, and Thomas Gorges. 2008. Reducing false positives by combining abstract interpretation and bounded model checking. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 188–197.
 - [47] Ze Sheng, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang. 2025. Llms in software security: A survey of vulnerability detection techniques and insights. *Comput. Surveys* 58, 5 (2025), 1–35.
 - [48] Alexey Shestov, Anton Cheshkov, Rodion Levichev, Ravil Mussabayev, Pavel Zadorozhny, Evgeny Maslov, Chibirev Vadim, and Egor Bulychev. 2024. Finetuning Large Language Models for Vulnerability Detection. CoRR abs/2401.17010 (2024). <https://doi.org/10.48550/ARXIV.2401.17010> arXiv:2401.17010
 - [49] Qingkai Shi, Rongxin Wu, Gang Fan, and Charles Zhang. 2020. Conquering the extensional scalability problem for value-flow analysis frameworks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 812–823.
 - [50] Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. 2018. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 693–706.
 - [51] Benjamin Jeremiah Steenhoek. 2024. *Understanding and improving deep learning models for vulnerability detection*. Ph.D. Dissertation. Iowa State University.
 - [52] Yulei Sui, Ding Ye, and Jingling Xue. 2012. Static memory leak detection using full-sparse value-flow analysis. In *Proceedings of the 2012 International Symposium on Software Testing and Analysis*. 254–264.
 - [53] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2023. When GPT Meets Program Analysis: Towards Intelligent Detection of Smart Contract Logic Vulnerabilities in GPTScan. arXiv:2308.03314 [cs.CR]
 - [54] Manuel Valdiviezo, Cristina Cifuentes, and Padmanabhan Krishnan. 2014. A method for scalable and precise bug finding using program analysis and model checking. In *Asian Symposium on Programming Languages and Systems*. Springer, 196–215.
 - [55] Carmine Vassallo, Sebastiano Panichella, Fabio Palomba, Sebastian Proksch, Harald C Gall, and Andy Zaidman. 2020. How developers engage with static analysis tools in different contexts. *Empirical Software Engineering* 25 (2020), 1419–1457.
 - [56] Chong Wang, Jianan Liu, Xin Peng, Yang Liu, and Yiling Lou. 2023. Boosting Static Resource Leak Detection via LLM-based Resource-Oriented Intention Inference. arXiv preprint arXiv:2311.04448 (2023).
 - [57] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLMDF: Analyzing Dataflow in Code with Large Language Models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
 - [58] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
 - [59] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically inspecting thousands of static bug warnings with large language model: How far are we? *ACM Transactions on Knowledge Discovery from Data* 18, 7 (2024), 1–34.
 - [60] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Tian Cong. 2024. Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We? *ACM Trans. Knowl. Discov. Data* 18, 7, Article 168 (jun 2024), 34 pages. <https://doi.org/10.1145/3653718>
 - [61] Fangzhou Wu, Qingzhao Zhang, Ati Priya Bajaj, Tiffany Bao, Ning Zhang, Ruoyu Wang, and Chaowei Xiao. 2023. Exploring the Limits of ChatGPT in Software Security Applications. CoRR abs/2312.05275 (2023). <https://doi.org/10.48550/ARXIV.2312.05275> arXiv:2312.05275
 - [62] Hua Yan, Yulei Sui, Shiping Chen, and Jingling Xue. 2018. Spatio-temporal context reduction: A pointer-analysis-based static approach for detecting use-after-free vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*. 327–337.
 - [63] Aidan Z. H. Yang, Claire Le Goues, Ruben Martins, and Vincent J. Hellendoorn. 2024. Large Language Models for Test-Free Fault Localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 17:1–17:12. <https://doi.org/10.1145/3597503.3623342>
 - [64] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. arXiv preprint arXiv:2210.03493 (2022).
 - [65] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2025. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–31.
 - [66] Xin Zhou, Ting Zhang, and David Lo. 2024. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions (ICSE-NIER'24). Association for Computing Machinery, New York, NY, USA, 47–51. <https://doi.org/10.1145/3639476.3639762>
 - [67] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51.
 - [68] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d'Alché-Buc, Emily B. Fox, and Roman Garnett (Eds.). 10197–10207. <https://proceedings.neurips.cc/paper/2019/hash/49265d2447bc3bbfe9e76306ce40a31f-Abstract.html>